

Arthur Lorentz, Lucas Bertol, Otávio Zimmer e Renato Marquioro



TRABALHO DE FUNDAMENTOS DE BANCO DE DADOS

PLATAFORMA STEAM

Escolha do Dataset

Foi selecionado um dataset aberto contendo informações de jogos da plataforma Steam, exibindo os 500 jogos com maior média de proprietários estimados (estimated owners).

Esse conjunto de dados reúne informações relevantes como nome do jogo, data de lançamento, número estimado de proprietários, avaliações, preços, etc.

A escolha desse dataset foi motivada pela sua riqueza de atributos, estrutura consistente em formato CSV e facilidade de adaptação para modelagem em banco de dados relacional, permitindo a aplicação completa dos conceitos de ETL, normalização e engenharia de dados.



01

Plataforma digital líder na distribuição de jogos para PC, com uma das maiores bibliotecas de títulos do mundo.

02

Grande volume de dados públicos sobre jogos, incluindo estatísticas de jogadores, preços e avaliações.

Etapa 1: Criação de um banco de dados desnormalizado

Solução

01

Escolha da fonte de dados

- Dataset aberto obtido da plataforma Steam
- Contém informações de jogos em formato CSV
- Selecionado por possuir grande volume e variedade de atributos

03

Geração do script SQL

- Criação do banco de dados no MySQL
- Definição da tabela contendo todas as colunas do dataset
- Elaboração do comando CREATE TABLE correspondente

02

Estruturação do banco de dados

- Análise das colunas do arquivo CSV
- Criação de um modelo relacional inicial (tabela desnormalizada)
- Definição dos tipos de dados (INT, VARCHAR, DATE, etc.)

04

Execução do script

- Execução do script no MySQL Workbench
- Criação do banco e estrutura de tabelas
- Preparação do ambiente para importação dos dados

Etapa 1: Criação de um banco de dados desnormalizado - especificação

Solução

Escolha da fonte de dados

- Dataset aberto da plataforma Steam (Kaggle)
- 500 jogos com maior nº estimado de proprietários
- Formato CSV com 40 colunas — fonte não-nacional

Banco e tabela criados

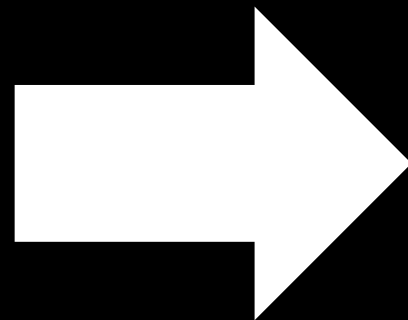
- Banco: trabalho_final_fdb_desnormalizado
- 1 única tabela: games
- 40 colunas — todas as colunas do CSV
- Sem chave primária, sem relacionamentos
- Colunas multivaloradas armazenadas como TEXT (ex: developers, genres, tags)

Tipos de dados definidos

app_id BIGINT, name VARCHAR(500), release_date VARCHAR(50), estimated_owners VARCHAR(50), price DECIMAL(10,2), windows / mac / linux BOOLEAN, about_the_game, reviews... TEXT, developers, publishers, genres, tags, categories, screenshots, movies, supported_language - TEXT. Colunas em amarelo contêm múltiplos valores — serão normalizadas na Etapa 2.

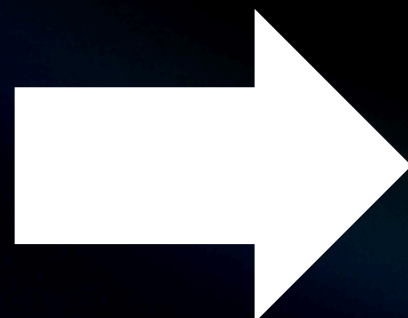
Etapa 2: Normalização do esquema desnormalizado

Aplicação de
formas
normais



Criação do
modelo
normalizado

Geração do
script SQL



Execução
do script

Ao todo, foram criadas 16 tabelas normalizadas

Solução

Normalização de Gêneros

Antes:

genres TEXT

Depois:

genre_id INT
name VARCHAR(100)

Etapa 2: Normalização do esquema desnormalizado - especificação

Solução

Listas simples

genres, tags, categories,
developers, publishers

Antes:

genres TEXT
"Action, Free to Play"

Depois:

genres(genre_id, name)
game_genres(app_id, genre_id)

Tabela dimensão + associativa
N:M para cada conjunto de
valores.

Formato Especial

supported_languages,
full_audio_languages

Antes:

['English', 'Portuguese
- Brazil', 'Japanese']

Depois:

languages (language_id, name)
game_supported_languages
game_audio_languages

Uma única tabela languages
compartilhada por idiomas
suportados e idiomas com áudio
completo.

Etapa 2: Normalização do esquema desnormalizado - especificação

Solução

Campo → dois atributos

estimated_owners

Antes:

estimated_owners VARCHAR
"10000000 - 20000000"

Depois:

estimated_owners_min INT
estimated_owners_max INT

O campo de faixa textual foi decomposto em dois atributos inteiros distintos.

Refinamento de Tipos

release_date, required_age, app_id,
e outros

Antes:

release_date VARCHAR(50),
required_age DECIMAL(5,2), sem
chave primária

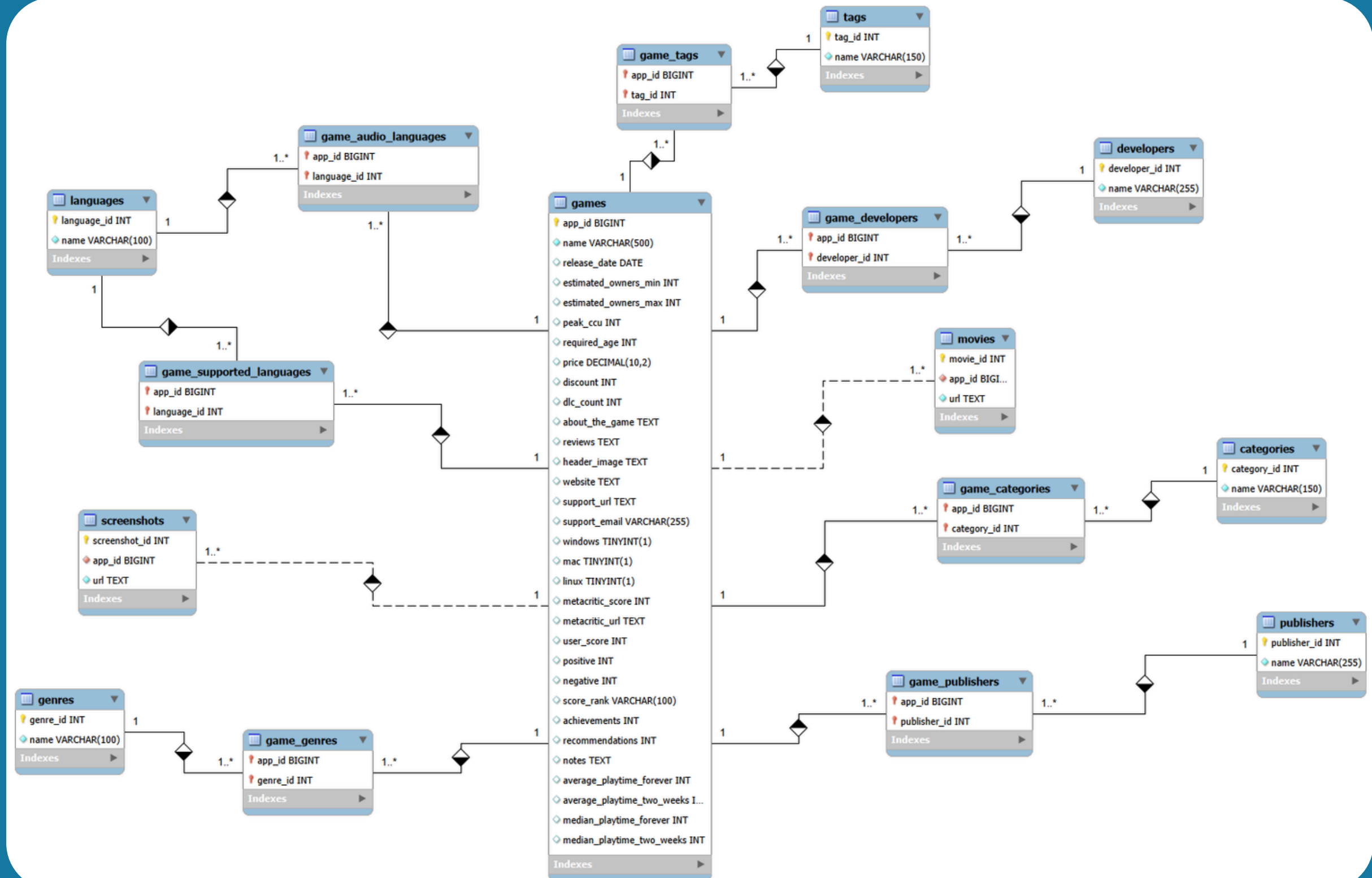
Depois:

release_date DATE, required_age
INT, app_id BIGINT PRIMARY KEY,
campos com DEFAULT e NOT NULL

Tipos ajustados para refletir melhor
a natureza dos dados e garantir
integridade.

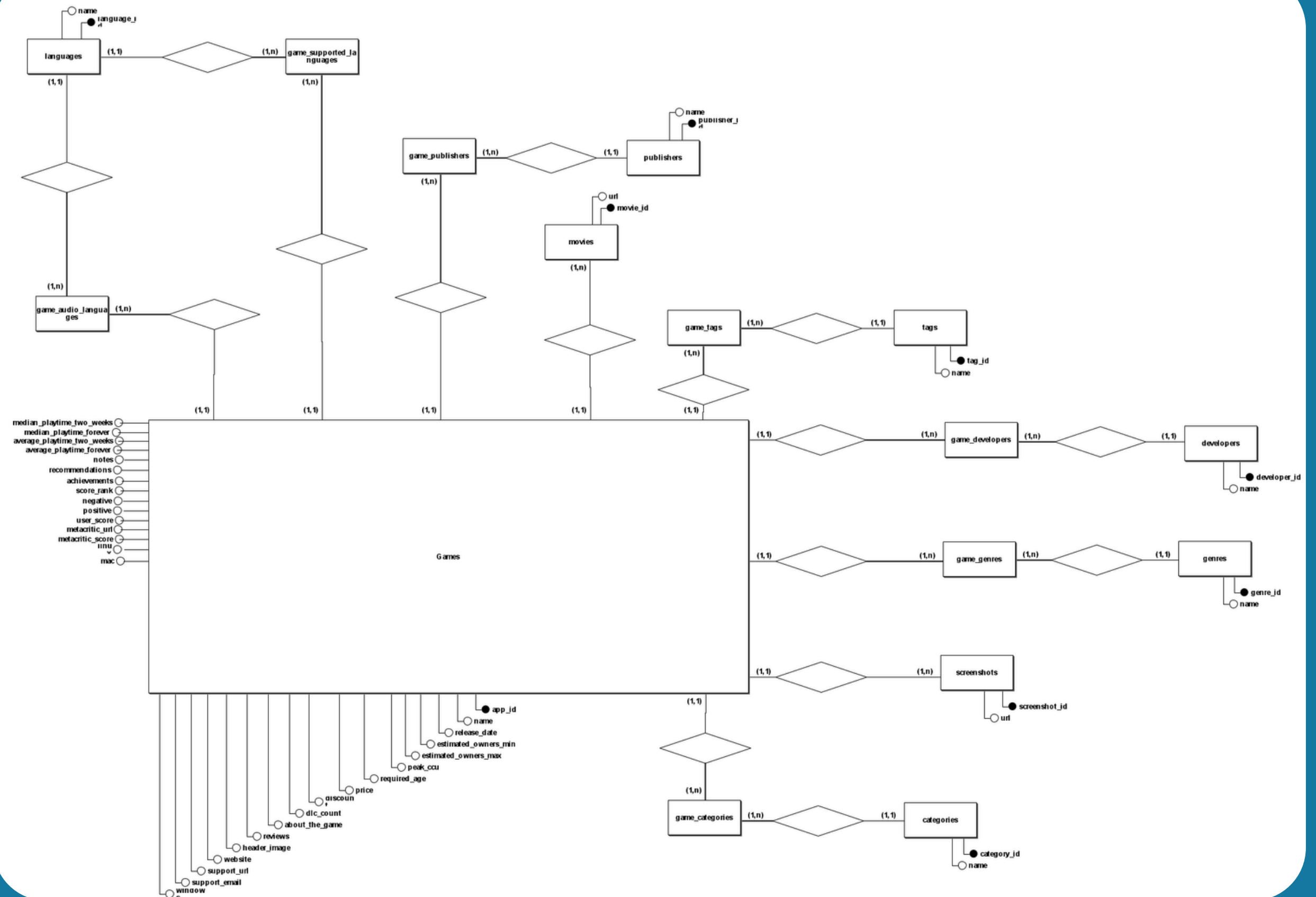
Etapa 3: Modelos conceituais

Solução



Etapa 3: Modelos conceituais

Solução



Etapa 3: Modelos conceituais

Solução

games(
 app_id, name, release_date, estimated_owners_min, estimated_owners_max,
 peak_ccu, required_age, price, discount, dlc_count, about_the_game,
 reviews, header_image, website, support_url, support_email, windows, mac,
 linux, metacritic_score, metacritic_url, user_score, positive, negative,
 score_rank, achievements, recommendations, notes, average_playtime_forever,
 average_playtime_two_weeks, median_playtime_forever, median_playtime_two_weeks)

developers(developer_id, name)

game_developers(app_id, developer_id)
 app_id referencia games
 developer_id referencia developers

publishers(publisher_id, name)

game_publishers(app_id, publisher_id)
 app_id referencia games
 publisher_id referencia publishers

genres(genre_id, name)

game_genres(app_id, genre_id)
 app_id referencia games
 genre_id referencia genres

categories(category_id, name)

game_categories(app_id, category_id)
 app_id referencia games
 category_id referencia categories

tags(tag_id, name)

game_tags(app_id, tag_id)
 app_id referencia games
 tag_id referencia tags

languages(language_id, name)

game_supported_languages(app_id, language_id)
 app_id referencia games
 language_id referencia languages

game_audio_languages(app_id, language_id)
 app_id referencia games
 language_id referencia languages

screenshots(screenshot_id, app_id, url)
 app_id referencia games

movies(movie_id, app_id, url)
 app_id referencia games

Etapa 4: Carga dos dados para o esquema normalizado

Solução

1

Extração

Os dados do arquivo games.csv foram importados automaticamente para uma tabela desnormalizada, por um script em Python preservando todas as colunas originais do dataset.

2

Transformação

Script SQL foi utilizado para aplicar a normalização dos dados. A lógica realiza a quebra de listas de textos separadas por vírgulas vindas do banco desnormalizado, isolando termos individuais.

3



Carga

Após a normalização, os dados foram inseridos nas tabelas relacionais do banco, utilizando chaves primárias e estrangeiras para garantir a integridade dos relacionamentos.

Etapa 4: Carga dos dados para o esquema normalizado

Solução

Extração

- Script Python lê o arquivo games.csv e insere todos os 500 registros na tabela desnormalizada preservando todas as colunas originais
- Tratamento de tipos: clean_bool, clean_int, clean_decimal, clean_str
- TRUNCATE da tabela antes de inserir, evitando duplicações

Transformação

- Script SQL lê a tabela desnormalizada e aplica a normalização, separando os campos multivalorados em suas respectivas tabelas
- Quebra de listas por vírgula com SUBSTRING_INDEX
- Limpeza de colchetes/aspas das linguagens



Carga

- Dados inseridos nas 16 tabelas normalizadas com chaves primárias e estrangeiras, garantindo integridade referencial.
- INSERT IGNORE evita duplicações
- Carga 1 (100%) — método totalmente automatizado

Etapa 5: Consultas sobre o esquema normalizado

Solução

Consulta 1: Jogos acima da média de avaliações positivas.

```
SELECT g.name, g.positive
FROM games g
WHERE g.positive >
(
  SELECT AVG(g.positive)
  FROM games g
)
ORDER BY g.positive DESC
LIMIT 20;
```


Etapa 5: Consultas sobre o esquema normalizado

Solução

Consulta 2: Listar jogos e suas empresas publicadoras, incluindo jogos sem publisher cadastrado.

```
SELECT g.name AS jogo,  
       p.name AS publisher  
FROM games g  
LEFT JOIN game_publishers gp  
  ON g.app_id = gp.app_id  
LEFT JOIN publishers p  
  ON gp.publisher_id = p.publisher_id  
ORDER BY g.name ASC;
```


Etapa 5: Consultas sobre o esquema normalizado

Solução

Consulta 3: Descobrir quais são os 10 gêneros mais lucrativos na plataforma, considerando apenas jogos que possuem desenvolvedores associados.

```
SELECT
  g.name AS genero,
  COUNT(DISTINCT gam.app_id) AS total_jogos,
  ROUND(AVG(gam.price), 2) AS preco_medio,
  SUM(gam.price * gam.estimated_owners_min) AS receita_minima_estimada
FROM genres g
JOIN game_genres gg ON g.genre_id = gg.genre_id
JOIN games gam ON gg.app_id = gam.app_id
JOIN game_developers gd ON gam.app_id = gd.app_id
GROUP BY g.genre_id, g.name
HAVING COUNT(DISTINCT gam.app_id) >= 5
ORDER BY receita_minima_estimada DESC
LIMIT 10;
```


Etapa 5: Consultas sobre o esquema normalizado

Solução

Consulta 4: Descobrir quais desenvolvedores possuem os jogos com as maiores médias de avaliações positivas.

```
SELECT
  d.name AS desenvolvedor,
  COUNT(g.app_id) AS total_jogos_lancados,
  SUM(g.positive) AS total_avaliacoes_positivas,
  ROUND(AVG(g.positive), 0) AS media_positivas_por_jogo
FROM developers d
JOIN game_developers gd ON d.developer_id = gd.developer_id
JOIN games g ON gd.app_id = g.app_id
GROUP BY d.developer_id, d.name
HAVING total_jogos_lancados >= 2
ORDER BY media_positivas_por_jogo DESC
LIMIT 10;
```


Etapa 5: Consultas sobre o esquema normalizado

Solução

Consulta 5: Listar os 15 jogos com preço igual ou superior a US\$ 25 e pico de jogadores simultâneos acima da média.

```
SELECT
  g.app_id,
  g.name,
  g.price,
  g.peak_ccu
FROM games g
WHERE g.price >= 25
  AND g.peak_ccu > (
    SELECT AVG(peak_ccu)
    FROM games
  )
ORDER BY g.peak_ccu DESC
LIMIT 15;
```


Etapa 5: Consultas sobre o esquema normalizado

Solução

Consulta 6: Exibe os desenvolvedores que possuem pelo menos um jogo com número de jogadores simultâneos acima da média da plataforma.

```
SELECT DISTINCT
  d.name AS desenvolvedor
FROM developers d
JOIN game_developers gd ON d.developer_id = gd.developer_id
JOIN games g ON gd.app_id = g.app_id
WHERE g.app_id IN (
  SELECT app_id
  FROM games
  WHERE peak_ccu > (
    SELECT AVG(peak_ccu)
    FROM games
  )
)
ORDER BY desenvolvedor;
```


Etapa 5: Consultas sobre o esquema normalizado

Solução

Consulta 7: Encontrar as empresas relacionadas a jogos muito populares.

```
SELECT
  d.name AS empresa,
  'Developer' AS tipo,
  g.name AS jogo,
  g.peak_ccu
FROM developers d
JOIN game_developers gd ON d.developer_id = gd.developer_id
JOIN games g ON gd.app_id = g.app_id
WHERE g.peak_ccu > 100000
UNION
SELECT
  p.name AS empresa,
  'Publisher' AS tipo,
  g.name AS jogo,
  g.peak_ccu
FROM publishers p
JOIN game_publishers gp ON p.publisher_id = gp.publisher_id
JOIN games g ON gp.app_id = g.app_id
WHERE g.peak_ccu > 100000

ORDER BY peak_ccu DESC;
```


OBRIGADO!